ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 28

Подготовка к контрольной работе №5

В рамках данного практического занятия предусмотрена разработка и защита итогового проекта — Контрольной работы №5.

Содержание работы

Контрольная работа №5 — самостоятельно реализованный полноценный веб-проект, демонстрирующий навыки фронтенд- и бэкенд-разработки, накопленные в ходе курса.

Общие требования к проекту

Независимо от выбранной темы проект должен соответствовать следующим техническим требованиям:

Требование	**Описание**
Backend	Express API с подключённой базой данных (SQL или NoSQL)
Frontend	React или Vue SPA (Single Page Application)
Авторизация	JWT + RBAC (Role-Based Access Control)
Контейнеризация	Docker + `docker-compose.yml` для запуска всего стека одной командой
Контроль версий	Git-репозиторий с осмысленными commit-сообщениями
Документация	`README.md` с описанием проекта и инструкцией по запуску
Тесты	Покрытие кода тестами не менее 50%
Развёртывание	Приложение запускается на VPS, в облаке или локально через `docker-compose`

Структура README.md

Репозиторий обязательно должен содержать `README.md` со следующими разделами:

```
# Название проекта

## Описание
Краткое описание проекта и его функциональности.

## Стек технологий
- Frontend: React / Vue
- Backend: Node.js, Express
- База данных: PostgreSQL / MongoDB
- Авторизация: JWT
- Контейнеризация: Docker, Docker Compose

## Запуск проекта

### Требования
- Docker и Docker Compose

### Шаги
1. Клонировать репозиторий: `git clone <url>`
2. Скопировать файл переменных окружения: `cp .env.example .env`
3. Запустить все сервисы: `docker compose up --build`
4. Открыть в браузере: http://localhost:3000

## Переменные окружения
Описание всех переменных из `.env.example`.

## Запуск тестов
`npm test` или `npm run test:coverage`
```

Варианты проектов

Необходимо выбрать **один** из предложенных вариантов или предложить свою.

Вариант 1. Лендинг (PWA + SEO)

Адаптивный лендинг для IT-курса или продукта с высоким показателем качества по метрике Lighthouse.

Функциональные требования:

- адаптивная вёрстка (mobile-first);

- поддержка режима PWA (Progressive Web App) через Service Worker: приложение должно открываться без интернета;
- SEO-оптимизация: корректные `<meta>`-теги, `<title>`, `canonical`, Open Graph;
- итоговый Lighthouse score > 90 по всем категориям (Performance, Accessibility, Best Practices, SEO).

Рекомендуемый стек: Vite + React (или Next.js для SSR/SSG), Workbox для Service Worker.

Вариант 2. Социальная сеть (Real-time + WebSocket)

Платформа с публикацией постов, комментариями и real-time уведомлениями.

Функциональные требования:

- регистрация и авторизация (JWT + RBAC: роли `user` и `admin`);
- создание, редактирование и удаление постов;
- комментарии к постам;
- лайки;
- real-time уведомления о новых лайках и комментариях через WebSocket;
- кэширование ленты в Redis;
- бесконечная прокрутка (infinite scroll) для ленты постов.

Рекомендуемый стек: React, Express, PostgreSQL, Redis, Socket.IO.

Вариант 3. E-commerce (Stripe + инвентарь)

Интернет-магазин с каталогом товаров, корзиной и приёмом платежей.

Функциональные требования:

- регистрация и авторизация (JWT + RBAC: роли `customer` и `admin`);
- каталог товаров с фильтрацией и поиском;
- корзина покупок (localStorage + синхронизация с сервером);
- оформление заказа с интеграцией Stripe для приёма платежей;
- управление инвентарём (количество товаров на складе);
- история заказов для пользователя;
- панель администратора для управления товарами.

Рекомендуемый стек: React, Express, PostgreSQL, Stripe API.

Вариант 4. Мессенджер (Encryption + offline sync)

Приложение для переписки с end-to-end шифрованием и работой в офлайн-режиме.

Функциональные требования:

- регистрация и авторизация (JWT);
- личная переписка 1-на-1;
- end-to-end шифрование сообщений (TweetNaCl.js);
- офлайн-режим: непрочитанные сообщения кэшируются через LocalStorage и Service Worker;
- индикатор печатания (typing indicator);
- статус «прочитано» (seen status).

Рекомендуемый стек: React, Express, MongoDB, Socket.IO, TweetNaCl.js.

Вариант 5. Менеджер задач (Real-time collaboration)

Доска задач с совместным редактированием в реальном времени.

Функциональные требования:

- регистрация и авторизация (JWT + RBAC: роли `owner`, `editor`, `viewer`);
- создание проектов и задач (Kanban-доска: «Новые» → «В работе» → «Готово»);
- real-time обновление доски при изменении задач другими пользователями (WebSocket);
- история изменений (undo/redo) с использованием паттерна Command;
- разрешение конфликтов при одновременном редактировании (CRDT или простая блокировка).

Рекомендуемый стек: React, Express, PostgreSQL, Socket.IO.

Вариант 6. AI-интеграция (OpenAI + RAG)

Чат-приложение с интеграцией ChatGPT и поиском по собственной базе знаний.

Функциональные требования:

- регистрация и авторизация (JWT);
- чат-интерфейс с потоковыми ответами ChatGPT (streaming);
- загрузка документов пользователем (PDF, TXT);
- векторный поиск по загруженным документам (RAG);

- история диалогов.

Рекомендуемый стек: React, Express, PostgreSQL + pgvector, OpenAI API.

Задание

1. Выберите один из шести вариантов проекта или свою тематику.
2. Реализуйте проект в соответствии с требованиями.
3. Протестируйте работоспособность всех функциональных возможностей.
4. Подготовьте корректный `README.md` с описанием проекта и инструкцией по запуску.
5. Проверьте, что репозиторий является открытым и приложение запускается командой `docker compose up`.

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой. Ссылка подгружается в соответствующий раздел СДО: Задания текущего контроля -> Контрольная работа №5.